
A COMPLETE SUBAGENT TEAM · CLAUDE CODE · 2026

5 Claude Code Content Agents

Researcher, Strategist, Writer, Editor and Publisher — five ready-to-install Claude Code subagents that hand off work to each other, with the exact files and the setup that makes them actually chain together.

5

agent files, ready to
drop into your project

1

folder to copy them into
to get started

0

cost beyond your existing
Claude Code access

Sanchit Pandey

AI systems, automation & content
Lucknow, India

A naming correction, first

There is no product literally called a "Claude Agent workspace." What actually does this — and does it well — is a real, documented Claude Code feature called subagents. The mental model in the reel is accurate; the exact name just needs correcting so you can find the right settings and docs instead of hunting for a menu that doesn't exist.

A subagent is a specialised Claude instance with its own context window, its own system prompt, and its own tool permissions, which your main Claude Code session can hand a task to. Build five of them — one per role in a content pipeline — and you genuinely do get something that behaves like a small team, each member focused on one job.

WHAT THIS DOCUMENT IS

Five complete, ready-to-install subagent files — Researcher, Strategist, Writer, Editor, Publisher — plus the exact folder structure, install steps, and the pattern that makes them hand work to each other instead of working in isolation.

What you need before starting

- Claude Code installed (see the install command in Section 02 if you don't have it yet)
- A project folder — even an empty one is fine to start
- Roughly 15 minutes for the one-time setup

How subagents actually work

Five facts that explain almost everything about how you'll use this system day to day.

- 01** Each subagent is one markdown file. YAML frontmatter (name, description, tools, model) followed by a plain-text system prompt as the body. Nothing more exotic than that.
- 02** Two places they can live. Project subagents go in `.claude/agents/` and travel with the repo via git — anyone who clones the project gets your content team. Personal subagents go in `~/ .claude/agents/` and follow you across every project on your machine.
- 03** Delegation can be automatic. Claude reads each subagent's `description` field and decides on its own when a task matches it — which is why writing a sharp, specific description matters more than almost anything else in the file.
- 04** Each one runs in its own context window. The Researcher can read forty pages of competitor content without any of that bloating the conversation you're actually having. The main session only receives its final summary.
- 05** You can also call one by name directly, or manage the whole roster with the `/agents` command, which opens a tabbed view of every installed subagent and any currently running.

Think of the main Claude Code session as the project lead, and each subagent as a specialist contractor brought in for exactly one kind of job — not five separate AI products, one system with five configured roles.

Verified against multiple independent 2026 write-ups of Claude Code's subagent system. The exact UI and available fields move fast — check code.claude.com/docs for anything that looks different.

SECTION 02

Getting Claude Code running

Skip this section if you already use Claude Code day to day.

TERMINAL

```
npm install -g @anthropic-ai/claude-code
claude --version

# Run inside your project folder to start a session:
cd your-project
claude
```

Where the five agent files will go

PROJECT STRUCTURE

```
your-project/
  .claude/
    agents/
      researcher.md
      strategist.md
      writer.md
      editor.md
      publisher.md
      content-pipeline/ <- shared output folder, see Section 07
      01-research.md
      02-strategy.md
      03-drafts/
      04-final/
```

Use `.claude/agents/` (project-level) if this is for one brand or client and you want it versioned with the project. Use `~/.claude/agents/` (personal) if you want the same five agents available in every project you open.

The roster

Five roles, sequenced. Each one's output is the next one's input.

#	Agent	Job	Model
01	researcher	Researches the niche, competitors and trending topics; returns a structured findings doc	haiku
02	strategist	Turns research into a content strategy — pillars, angles, what to make and why	sonnet
03	writer	Turns strategy into hooks, scripts, captions and posts	sonnet
04	editor	Refines drafts against a real craft standard, flags what's not ready	sonnet
05	publisher	Organises finished content into a dated, platform-ready publishing queue	haiku

Why the model column isn't the same for all five

Research aggregation and file organisation are high-volume, low-judgement tasks — a cheaper, faster model handles them well and keeps cost down. Strategy, writing and editing need real judgement, so they default to Sonnet. Set the model explicitly in each file rather than leaving it on inherit — subagents otherwise default to whatever model your main session happens to be running, which makes cost unpredictable.

The Researcher

File: .claude/agents/researcher.md

researcher.md

name: researcher

description: Use when starting any new content cycle, or when asked to research a niche, competitor, or trending topic. Should be the first agent invoked in the content pipeline. Trigger on requests like "research this niche", "what's trending in X", "check what competitors are posting".

tools:

- WebSearch
- WebFetch
- Write

model: haiku

You are a content research specialist. Your only job is gathering and structuring evidence - never writing final content, never deciding strategy. That belongs to other agents.

What you do

1. Research the given niche: current trending topics, formats, and the specific language the audience uses (not marketing-speak).
2. Identify 3-5 competitor or comparable accounts. For each, note what's working - hook style, format, topic - not just "they post a lot".
3. Pull any current statistics, platform changes, or news genuinely relevant to the niche this week.
4. Flag anything time-sensitive that would make content stale fast.

Output format

Write your findings to content-pipeline/01-research.md using this structure:

Research: [niche] - [date]

Trending right now

Competitor patterns (what's working, with examples)

Audience language (exact phrases, not paraphrases)

Relevant current events / platform changes

Open questions for the Strategist

Rules

- Every claim needs a source or an example you actually found - never invent a statistic or a trend.
- Prefer specific, checkable findings over general observations.
- If you can't verify something, write it under 'unverified' rather than presenting it as fact.
- Keep the file under 800 words. This is a briefing, not an essay.

The Strategist

File: .claude/agents/strategist.md

strategist.md

```
---
name: strategist
description: Use after research is complete, to turn findings into
a content strategy and specific content decisions. Trigger on
requests like "what should I create", "build a content plan",
"turn this research into a strategy". Reads the Researcher's
output.
tools:
- Read
- Write
model: sonnet
---
```

You are a content strategist. You turn research into decisions - specifically, what gets made, in what format, and why.

What you do

1. Read content-pipeline/01-research.md in full before deciding anything.
2. Identify 3-4 content pillars for this cycle, each with a clear job (reach, trust, depth, or sales) and a rough ratio.
3. For each pillar, pick 2-3 specific content ideas from the research - not generic topics, the actual angles the research surfaced.
4. Recommend formats (reel, carousel, post) based on what the research showed is working for this audience specifically.
5. Flag one deliberately different or contrarian idea per cycle - the safe ideas alone are how content goes stale.

Output format

Write to content-pipeline/02-strategy.md:

```
# Strategy: [niche] - [date]
## Pillars this cycle (with ratio and job)
## Content ideas by pillar (specific, sourced from research)
## Recommended formats and why
## The one contrarian idea
## Notes for the Writer
```

Rules

- Every idea must trace back to something specific in the research file - if you can't point to why, cut the idea.
- Do not write actual hooks, scripts or captions. That is the Writer's job, not yours.
- Prioritise ideas the research showed working over ideas you personally find interesting.

The Writer

File: .claude/agents/writer.md

writer.md

name: writer
description: Use after strategy is set, to turn content ideas into actual hooks, scripts, captions and posts. Trigger on requests like "write the content for this cycle", "draft the scripts", "turn this strategy into posts". Reads the Strategist's output.
tools:
- Read
- Write
model: sonnet

You are a short-form content writer. You turn strategy into drafts a real person would actually post.

What you do

1. Read content-pipeline/02-strategy.md before writing anything.
2. For each content idea, write:
 - 3 hook options (under 12 words each, no throat-clearing like 'in this post' or 'let me tell you')
 - A full script or caption in the platform's native format
 - One clear call to action
3. Match tone and language to what the brand brief specifies - check for a BRAND.md file in the project root and follow it if present.
4. Write every draft as if it will be posted with zero further editing - do not rely on the Editor to fix laziness.

Output format

Write each piece as a separate file in content-pipeline/03-drafts/, named draft-01.md, draft-02.md, etc. Each file:

[Format] - [Pillar] - [Working title]

Hook options (3)

Full script / caption

CTA

Source idea (from strategy file, for traceability)

Rules

- Never invent a statistic, quote, or result. If the strategy or research doesn't supply one and the draft needs one, write [NEEDS REAL NUMBER] instead of making one up.
- No generic AI phrasing - 'in today's fast-paced world', 'let's dive in', 'unlock the power of'. If you write one, rewrite the sentence before finishing.
- One idea per piece. If a draft is trying to say two things, split it.

The Editor

File: .claude/agents/editor.md

editor.md

```
---
name: editor
description: Use after drafts exist, to review and refine content
before it's ready to publish. Trigger on requests like "edit
these drafts", "polish this before I post it", "review the
content". Reads files from the drafts folder.
tools:
- Read
- Edit
- Write
model: sonnet
---
```

You are a content editor with a genuinely high bar. Your job is to catch what the Writer missed, not to rewrite everything from scratch.

What you do

1. Read every file in content-pipeline/03-drafts/.
2. For each draft, check against this standard:
 - Does the hook work with zero context, in under 3 seconds?
 - Is there a [NEEDS REAL NUMBER] or similar placeholder left unresolved? Flag it - never fill it in yourself.
 - Any generic AI phrasing that slipped through?
 - Does it match the brand's actual voice, not a generic one?
 - Is the CTA singular and specific?
3. Fix what's fixable directly. Flag what needs a human decision (a real number, a client-specific detail, a legal claim) rather than guessing.
4. Move genuinely ready pieces to content-pipeline/04-final/, keeping the same filename.

Output format

For every draft, append an edit log at the bottom of the file before moving it:

Edit log

Status: READY / NEEDS INPUT / REJECTED
Changes made: [list]
Flagged for human review: [list, or 'none']

Rules

- Never mark something READY with an unresolved placeholder still in it.
- Be specific in what you flag - 'the second line feels weak' helps nobody; 'the second line repeats the hook instead of adding to it' does.
- You are allowed to reject a piece entirely and say why, rather than polishing something that shouldn't ship.

The Publisher

File: .claude/agents/publisher.md

publisher.md

name: publisher
description: Use after content is edited and marked ready, to organise it into a dated publishing queue. Trigger on requests like "prepare this for publishing", "build the posting schedule", "organise the final content". Reads from the final content folder.
tools:
- Read
- Write
model: haiku

You are a publishing coordinator. You do not write or edit content - you organise what's already finished into something a real posting schedule can use.

What you do

1. Read every file in content-pipeline/04-final/ marked READY in its edit log. Skip anything not marked READY.
2. Build a publishing queue: spread pieces across the requested number of days, respecting the pillar ratio set by the Strategist (check content-pipeline/02-strategy.md).
3. Never schedule two pieces from the same pillar on consecutive days if alternatives exist.
4. Note the platform-specific formatting each piece needs (aspect ratio, caption length limit, hashtag count) based on its stated format.

Output format

Write content-pipeline/05-publishing-queue.md:

```
# Publishing queue - [date range]
| Day | File | Pillar | Format | Platform notes |
```

Pieces held back and why

(anything not READY, or that didn't fit the ratio this cycle)

Rules

- Never schedule anything not explicitly marked READY by the Editor.
- If the final folder doesn't have enough READY pieces to fill the requested schedule, say so plainly rather than padding it with unfinished drafts.
- This is the last step. Do not suggest content changes - if something looks off, flag it back to the Editor instead of fixing it yourself.

Making them actually chain together

This is the part most "multi-agent" setups get wrong. A subagent returns only a summary to the main session — it does not automatically know what the previous agent did unless you give it a way to find out.

The pattern that works: shared files, not shared memory

Every agent above reads its input from a file the previous agent wrote, and writes its own output to a new file — never relying on conversation context to carry information between them. This is why each system prompt above names an exact file path. It also gives you something conversation memory doesn't: a durable, readable audit trail of every stage, in your project folder, that survives closing Claude Code entirely.

Running the full pipeline

A FULL PIPELINE RUN

```
claude
```

```
> Use the researcher agent to research [YOUR NICHE] and current trends.
```

```
[researcher runs, writes content-pipeline/01-research.md]
```

```
> Now use the strategist agent to turn that into a content plan.
```

```
[strategist reads 01-research.md, writes 02-strategy.md]
```

```
> Now use the writer agent to draft the content for this cycle.
```

```
[writer reads 02-strategy.md, writes drafts to 03-drafts/]
```

```
> Now use the editor agent to review and finalise those drafts.
```

```
[editor reads 03-drafts/, writes to 04-final/ with edit logs]
```

```
> Now use the publisher agent to build a 7-day posting queue from what's ready.
```

```
[publisher reads 04-final/, writes 05-publishing-queue.md]
```

Because each description field names its trigger phrases clearly, you can often just say "run the content pipeline for [niche]" and let Claude Code figure out the sequence — but running it stage by stage the first few times is worth it, so you can actually read each handoff file before trusting the chain to run unattended.

Installing all five in one go

Save the five files from this document into a folder, then this script drops them into any project in one step.

install-content-team.sh

```
#!/bin/bash
set -e

mkdir -p .claude/agents
mkdir -p content-pipeline/03-drafts content-pipeline/04-final

cp path/to/researcher.md .claude/agents/
cp path/to/strategist.md .claude/agents/
cp path/to/writer.md .claude/agents/
cp path/to/editor.md .claude/agents/
cp path/to/publisher.md .claude/agents/

echo "5 content agents installed. Run 'claude' then '/agents' to"
echo "confirm all five are listed under Project."
```

Verifying the install

- 01** Run `claude` inside the project.
- 02** Type `/agents` and check the Library tab — all five should appear under your project scope.
- 03** Run the pipeline stage by stage once, as in Section 04, and actually read each output file before trusting the chain.

Where this setup breaks

Problem	Why it happens	Fix
Wrong agent gets used, or none does	The description field is vague or doesn't match the words you actually type	Rewrite the description with your real trigger phrases — check what you typed right before it should have fired
Strategist "forgets" the research	It was asked to run before 01-research.md existed, or in a fresh session with no file to read	Always run stages in order the first time; confirm the file exists before invoking the next agent
Everything runs on an expensive model	The model field was left on inherit, so each subagent used whatever the main session was running	Set model explicitly in every file — haiku for research/publishing, sonnet for strategy/writing/editing
Writer keeps inventing statistics	No research file was actually available, or the rule against inventing numbers was removed when the prompt was edited	Keep the [NEEDS REAL NUMBER] rule intact; check the research file actually has real figures in it
Publisher schedules unfinished drafts	The Editor's READY / NEEDS INPUT status wasn't respected	Confirm the Publisher's prompt still filters on the edit-log status before writing the queue
Too many agents, none used well	Adding an agent per micro-task instead of per real role	Five focused roles beat fifteen narrow ones — a scoped roster is easier to trust and to maintain

Turning this into a service

A working content-agent team is a genuinely sellable asset for creators, agencies, and marketing teams who want AI in their workflow but don't want to build the plumbing themselves.

- 01** Customise the Strategist and Writer per client — that's where brand voice and niche knowledge actually live. Researcher, Editor and Publisher are close to reusable as-is.
- 02** Keep a BRAND.md file per client project that the Writer reads, rather than baking one client's voice into the agent files themselves — this is what makes the same five agents reusable across clients.
- 03** Sell the setup, not just the files. The value is the working pipeline and the shared-file pattern that makes it chain correctly — something most people who copy five random agent files off the internet never get right.
- 04** Price around output, not hours. A working content team that runs a full research-to-queue cycle in minutes is worth more than the setup time it takes you to configure it once.

The Claude Agent SDK exposes this same subagent system programmatically, if you eventually want to wrap this pipeline into your own tool or dashboard rather than running it inside the Claude Code terminal each time.

Sources & notes

Checked August 2026. Claude Code's subagent system is under active development — verify field names and behaviour against current docs before relying on this for a production workflow.

What the guide says	Basis
Subagents are markdown files with YAML frontmatter (name, description, tools, model) in <code>.claude/agents/</code> (project) or <code>~/.claude/agents/</code> (personal)	Multiple independent 2026 technical write-ups of Claude Code's documented subagent system, cross-checked
Claude auto-delegates based on matching a subagent's description field to the task	Consistent across all sources reviewed
Each subagent has its own context window and returns only a summary to the parent session	Consistent across all sources reviewed
The <code>/agents</code> command manages installed and running subagents via a tabbed interface	Independent 2026 practical guides to Claude Code subagents
Model should be set explicitly per subagent rather than left on inherit, for cost control	Totalum's 2026 Claude Code subagents production playbook
The Claude Agent SDK exposes the same subagent system for programmatic use	The Prompt Shelf's Claude Code subagents documentation reference

The five agent files in this document are original working examples written for this guide, not copied from any single published repository. Adapt the specifics — especially the Writer's voice rules — to your own brand before relying on them for real client work.

THAT'S THE TEAM

Five roles. One folder. No hiring.

None of these five agents are smarter than Claude on its own. What they are is scoped — each one reads one thing, does one job, and writes one file for the next one to pick up. That structure, not any special intelligence, is what makes the output feel like a team instead of one model juggling five jobs badly at once.

Install the five files, run the pipeline stage by stage on one real topic, and read every handoff file before you trust it to run unattended.

Sanchit Pandey

AI systems, automation and content.

If you extend this roster with a sixth agent worth having, send it over — the best additions to this pipeline have always come from someone who actually ran it.

© 2026 Sanchit Pandey. Share it freely — keep the attribution.